

Cryptography and Network Security

Primality Testing

Primality Testing

- Introduction of Prime Numbers
- Power Algorithm
- Fermat Theorem for Primality testing
- Miller Rabin Testing for Primality

Introduction to Prime Numbers

- A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself.
- **Examples:** 2, 3, 5, 7, 11, 13, ...
- **Importance in Cryptography:**
 - **RSA Encryption:** Uses large primes to generate public and private keys.
 - **Field Size:** In finite field cryptography, larger primes provide stronger security.
- **Why are primes important in cryptography?**
 - Because factoring the product of two large primes is computationally hard, forming the basis of security in RSA.

Basic Primality Testing Method

Trial Division Method

Simplest method: Check divisibility from 2 to $\sqrt{n}$.

- Example: Is 29 prime?
Check: $29 \bmod 2 \neq 0$, $29 \bmod 3 \neq 0$, ... up to $\sqrt{29} \approx 5$.
No divisors found $\rightarrow$ 29 is prime.
- **Limitations:**
 - Inefficient for large numbers (e.g., 1000-digit numbers).
 - Time complexity: $O(\sqrt{n})$, which is too slow for cryptographic use.

Fermat's Primality Check

- **Fermat's Little Theorem** says:

- If n is **prime**, then for any integer a where $1 < a < n - 1$:

$$\text{If } n \text{ is a prime, then } a^{n-1} \equiv 1 \pmod{n}.$$

- This is **always true** for primes.

- **But the reverse is not always true:**

- If $a^{n-1} \equiv 1 \pmod{n}$, it **does not guarantee** n is prime.
- Some composite numbers can also satisfy this condition.

- **Key Takeaway:**

- Prime $\rightarrow$ Always passes Fermat's test.
- Composite $\rightarrow$ **Sometimes pass Fermat's test.**

Example

Example

Let's see for a composite number 561. Does it pass Fermat's test?
If it passes that means Composite sometimes pass Fermat's test.

Solution

Given: $n = 561$, test with **base** $a = 2$ ($1 < a < n - 1$)

Step 1: Compute $n - 1 = 560$

Step 2: We need $2^{560} \bmod 561$

Compute powers of 2 modulo 561:

$$\begin{aligned} 2^1 &= 2 & 2^2 &= 4 & 2^4 &= 16 & 2^8 &= 256 & 2^{16} &= 256^2 \bmod 561 = 460 & 2^{32} \\ &= 460^2 \bmod 561 = 103 & 2^{64} &= 103^2 \bmod 561 = 511 & 2^{128} &= 511^2 \bmod 561 \\ &= 256 & 2^{256} &= 256^2 \bmod 561 = 460 \end{aligned}$$

Example Continue...

Step 3: Combine for 2^{560} :

$$\begin{aligned} 2^{560} &= 2^{256} \times 2^{128} \times 2^{64} \times 2^{32} \times 2^{16} \times 2^4 \\ &= 460 \times 256 \times 511 \times 103 \times 460 \times 16 \pmod{561} \end{aligned}$$

Compute stepwise mod 561:

$$\begin{aligned} 460 \times 256 &\equiv 103 \\ 103 \times 511 &\equiv 511 \\ 511 \times 103 &\equiv 460 \\ 460 \times 460 &\equiv 103 \\ 103 \times 16 &\equiv 1648 \equiv 1 \pmod{561} \end{aligned}$$

✓ Final: $2^{560} \equiv 1 \pmod{561}$

→ **Passes test, even though composite.**

Fermat's Primality Check

Algorithm 1: Fermat Primality Test

Data: n : number to be tested for primality, k : number of times the test will be repeated for

Result: Composite or Pseudoprime

Function $isPrime(n, k)$ **is**

```
    repeat  $k$  times
        a  $\leftarrow$  uniform random number between 2 and  $n-1$ ;
        if  $a^{n-1} \not\equiv 1 \pmod{n}$  then
            return Composite;
        end
    end
    return Pseudoprime;
```

Pseudoprime means probable prime

Testing Fermat's Theorem Prime vs. Carmichael Numbers

Part 1: Testing a Real Prime $n = 23$

Step-by-Step Example:

Let's check if **23** (a known prime) satisfies Fermat's Little Theorem:

Theorem says:

If n is prime, then for any a where $1 < a < n - 1$:

$$a^{n-1} \equiv 1 \pmod{n}$$

Test with $a = 5$:

$$n = 23, n - 1 = 22$$

Compute $5^{22} \pmod{23}$

Using modular exponentiation:

$$5^2 = 25 \equiv 2 \pmod{23} \quad 5^4 \equiv 2^2 = 4 \quad 5^8 \equiv 4^2 = 16 \quad 5^{16} \equiv 16^2 = 256 \equiv 3 \pmod{23}$$

Now combine:

$$5^{22} = 5^{16} \times 5^4 \times 5^2 = 3 \times 4 \times 2 = 24 \equiv 1 \pmod{23}$$

Result: $5^{22} \equiv 1 \pmod{23} \rightarrow$ **Passes the test**

(Expected, since 23 is prime)

Testing Fermat's Theorem Prime vs. Carmichael Numbers

Part 2: However, when $n = 561$

It passes the Fermat's test. Such numbers are called Carmichael numbers

First few Carmichael numbers:

561, 1105, 1729, 2465, ...

561 is the **smallest** Carmichael number.

Cost of Fermat's Primality Test

The Cost of Fermat's Test

- The Problem is Repeated Power Operations

Fermat's Test:

To check if n is prime, we need to compute:

$$a^{n-1} \bmod n$$

for one or more values of a .

Why normal multiplication is costly?

Example: $n = 1000003$ (a large number)

We need to compute $a^{1000002} \bmod n$

Doing this by normal multiplication would require **over 1 million multiplications!**

That's **impractical** for large numbers used in cryptography.

Efficient Binary Exponentiation Algorithm

Instead of multiplying a by itself $n - 1$ times, we use a smarter method:

Binary Exponentiation (Fast Power Algorithm):

Convert the exponent $n - 1$ into **binary**.

Use **repeated squaring** to compute powers efficiently.

Example in a nutshell:

To compute a^{13} :

Write $13_{10} = 1101_2$ (8+4+1)

Combine: $a^{13} = a^8 \times a^4 \times a^1$

Example: To compute a^{13} :

Step 1: $a^1 = a$

Step 2: $a^2 = a \times a$

Step 3: $a^4 = a^2 \times a^2$

Step 4: $a^8 = a^4 \times a^4$

Now: $13 = 8 + 4 + 1$

So: $a^{13} = a^8 \times a^4 \times a^1$

Only 5 multiplications instead of 13!

Efficient Exponentiation Example

Example Compute $3^{74} \bmod 100$

We want to calculate:

$$3^{74} \bmod 100$$

Why not multiply 74 times?

That would take forever!

$3 \times 3 \times 3 \times \dots$ (74 times) $\rightarrow$ huge number!

The Smart Way: Repeated Squaring

Step 1: Find highest power

We need powers up to 3^{64} because 64 is the highest power of 2 ≤ 74 .

Step 2: Build the power table quickly:

We **square repeatedly** instead of multiplying:

$$3^1 = 3$$

$$3^2 = 3 \times 3 = 9$$

$$3^4 = 9 \times 9 = 81$$

$$3^8 = 81 \times 81 = 6561 \rightarrow 61 \pmod{100}$$

$$3^{16} = 61 \times 61 = 3721 \rightarrow 21 \pmod{100}$$

$$3^{32} = 21 \times 21 = 441 \rightarrow 41 \pmod{100}$$

$$3^{64} = 41 \times 41 = 1681 \rightarrow 81 \pmod{100}$$

Only 6 multiplications to get all these powers!

Efficient Exponentiation Example

Step 3: Write 74 in binary

$$74_{10} = 1001010_2$$

What this means:

$$74 = 64 + 8 + 2$$

So:

$$3^{74} = 3^{64} \times 3^8 \times 3^2$$

Step 4: Multiply together with mod 100:

$$3^{64} = 81$$

$$3^8 = 61$$

$$3^2 = 9$$

Multiply step-by-step:

$$\text{First: } 81 \times 61 = 4941 \rightarrow 41 \pmod{100}$$

$$\text{Then: } 41 \times 9 = 369 \rightarrow 69 \pmod{100}$$

Final Answer:

$$3^{74} \pmod{100} = 69$$

Efficient Exponentiation Example

Total Multiplications:

6 multiplications to build the power table

2 multiplications to combine them

Total: 8 multiplications

(Instead of 73 multiplications the normal way!)

Why This Matters in Cryptography:

Imagine computing $3^{1000000} \bmod n$:

Normal way: 999,999 multiplications (impossible!)

This way: Only about 40 multiplications (easy!)

This algorithm makes modern encryption possible!

Binary Modular Exponentiation Algorithm (From left to Right)

Input Parameters:

$a \rightarrow$ base number

$x \rightarrow$ exponent, expressed in binary as $(x_t x_{t-1} \dots x_0)$

$n \rightarrow$ modulus

Algorithm Steps:

Step 1: Initialize

$y := 1$

Step 2: Process Bits from Most Significant to Least Significant

for $i = t$ down_to 0 :

$y := y \times y \bmod n$ // Square step

if $x_i = 1$ then

$y := y \times a \bmod n$ // Multiply step

Step 3: Return Result

return y

Output:

$y = a^x \bmod n$

In Simple Words:

Step 1: Start with $\text{result} = 1$

Step 2: Convert exponent x to binary

Step 3: For each bit (left to right):

- **a.** Square: $\text{result} = \text{result} \times \text{result} \bmod n$

- **b.** If bit = 1:
 $\text{result} = \text{result} \times \text{base} \bmod n$

Step 4: Output result

Example

Example Compute a^{74} efficiently using the Binary Exponentiation algorithm.

Step 1: Convert 74 to binary

$$74_{10} = 1001010_2 \quad \text{i.e. } 64+8+2$$

Binary digits (from most to least significant):

$$x_6 = 1, x_5 = 0, x_4 = 0, x_3 = 1, x_2 = 0, x_1 = 1, x_0 = 0$$

Step 2: Algorithm Setup

Start with $y = 1$

Process bits from left (x_6) to right (x_0)

Apply:

Square step: $y = y \times y$

Multiply step: If bit = 1, then $y = y \times a$

Example

Execution Table

i	x_i	y after step	Explanation
—	—	1	Start with $y = 1$
6	1	a	Square: $1^2 = 1 \rightarrow$ Multiply by $a : 1 \times a = a$
5	0	a^2	Square: $a^2 = a^2 \rightarrow$ Skip (bit=0)
4	0	a^4	Square: $(a^2)^2 = a^4 \rightarrow$ Skip (bit=0)
3	1	a^9	Square: $(a^4)^2 = a^8 \rightarrow$ Multiply: $a^8 \times a = a^9$
2	0	a^{18}	Square: $(a^9)^2 = a^{18} \rightarrow$ Skip (bit=0)
1	1	a^{37}	Square: $(a^{18})^2 = a^{36} \rightarrow$ Multiply: $a^{36} \times a = a^{37}$
0	0	a^{74}	Square: $(a^{37})^2 = a^{74} \rightarrow$ Skip (bit=0)

$$a^{74} = a^{64} \times a^8 \times a^2$$

Our algorithm built these pieces:

a^2 after bit 5 (i.e. $i=5$)

a^8 while processing bit 3

a^{64} while processing bit 0

General Formula:

For exponent x , number of operations $\approx \log_2(x)$

Efficiency Comparison: Number of Multiplications for a^{74}

Normal	73
Binary Exponentiation	7

Exercise

1. Compute $3^{13} \bmod 7$ efficiently using the Binary Exponentiation algorithm. **Let's see numerical on board**
2. Compute $5^{88} \bmod 15$ efficiently using the Binary Exponentiation algorithm.

Properties of Prime Numbers

Property 1

If p is prime and a is a number between 1 and $p - 1$, then $a^2 \bmod p = 1$ *ONLY IF* $a \bmod p = 1$ *OR* $a \bmod p = -1$

Example: Let's take prime $p = 7$.

Check squares modulo 7:

$$1^2 = 1 \bmod 7 = 1 \checkmark (\mathbf{a \bmod 7 = 1})$$

$$2^2 = 4 \bmod 7 = 4$$

$$3^2 = 9 \bmod 7 = 2$$

$$4^2 = 16 \bmod 7 = 2$$

$$5^2 = 25 \bmod 7 = 4$$

$$6^2 = 36 \bmod 7 = 1 \checkmark (a \bmod 7 = 6, \text{ which is } \mathbf{-1 \bmod 7})$$

Only $a = 1$ and $a = 6$ yield a square congruent to 1 mod 7.

Property 2

Let p be a prime number greater than 2. Since p is odd, the value $p - 1$ is even. Therefore, we can write

$$p - 1 = 2^k \times q$$

where $k > 0$ and q is an odd number.

Examples:

For $p = 17$:

$$p - 1 = 17 - 1 = 16 \text{ (even)}$$

$$16 = 2^4 \times 1$$

So, $k = 4, q = 1$.

For $p = 29$:

$$p - 1 = 29 - 1 = 28$$

$$28 = 4 \times 7 = 2^2 \times 7$$

So, $k = 2, q = 7$.

Miller Rabin Test

Algorithm to check n is probably prime or not

Step 1: Factor $n - 1$

Write $n - 1$ as:

$$n - 1 = 2^k \cdot m$$

where: $k \geq 1$ (integer) m is odd

Step 2: Select a Random Base

Choose a random integer a such that:

$$1 < a < n - 1$$

Step 3: Compute Sequence of Powers

Let $b_0 = a^m \pmod{n}$

For $i = 1$ to k :

$$b_i = (b_{i-1})^2 \pmod{n}$$

Step 4: Check Conditions

If:

$$b_i \equiv 1 \pmod{n} \text{ and } b_{i-1} \not\equiv -1 \pmod{n}$$

Return "**Composite**"

Else:

$$b_i \equiv -1 \pmod{n}$$

Return "**Probably Prime**"

If **neither condition** holds:

→ Return "**Composite**"

Miller Rabin Test

Example Testing $n = 561$ for Primality

Step 1: Factor $n - 1$

$$n - 1 = 561 - 1 = 560$$

We write 560 as $2^k \cdot m$ with m odd:

Divide by 2 repeatedly:

$560 \div 2^1 = 280$ ($m = 280$, but m is even, , ***so continue till m is odd***)

$560 \div 2^2 = 140$ ($m = 140$, still even, continue)

$560 \div 2^3 = 70$ ($m = 70$, still even, continue)

$560 \div 2^4 = 35$ ($m = 35$, **now m is odd**)

So:

$$n - 1 = 560 = 2^4 \cdot 35$$

$$k = 4, m = 35$$

Step 2: Select a Random Base

Choose a with $1 < a < 560$.

Let's pick $a = 2$ (randomly chosen).

Miller Rabin Test

Step 3: Compute Sequence of Powers

$$\begin{aligned}\text{Compute } b_0 &= a^m \pmod{n} \\ b_0 &= 2^{35} \pmod{561}\end{aligned}$$

Compute using binary exponentiation:

$$\begin{aligned}\text{Write 35 in binary: } 35 &= 32 + 2 + 1 \\ \text{Or: } 35 &= 100011_2\end{aligned}$$

Build powers of 2 modulo 561:

$$1. 2^1 \pmod{561} = 2$$

$$2. 2^2 = 4 \pmod{561} = 4$$

$$3. 2^4 = 4^2 = 16 \pmod{561} = 16$$

$$4. 2^8 = 16^2 = 256 \pmod{561} = 256$$

$$\begin{aligned}5. 2^{16} &= 256^2 = 65536 \\ &561 \times 116 \Rightarrow (561 \times 100 \\ &= 56100 \text{ and } 561 \times 16 = 8976) \\ &56100 + 8976 = 65076 \\ &65536 - 65076 = 460 \\ \text{So: } 2^{16} &\pmod{561} = 460\end{aligned}$$

$$\begin{aligned}6. 2^{32} &= 460^2 = 211600 \\ &561 \times 377 \Rightarrow \\ &561 \times 300 = 168300 \\ &561 \times 70 = 39270 \\ &561 \times 7 = 3927\end{aligned}$$

$$\begin{aligned}\text{Sum: } 168300 + 39270 &= 207570 \\ 207570 + 3927 &= 211497 \\ 211600 - 211497 &= 103 \\ \text{So: } 2^{32} &\pmod{561} = 103\end{aligned}$$

Miller Rabin Test

Now combine for 2^{35} :

$$\begin{aligned} 2^{35} &= 2^{32} \times 2^2 \times 2^1 \\ &= 103 \times 4 \times 2 \pmod{561} \end{aligned}$$

Step-by-step multiplication:

$$\begin{aligned} 103 \times 4 &= 412 \\ 412 \times 2 &= 824 \end{aligned}$$

Since $824 > 561$:

$$824 - 561 = 263$$

Thus:

$$b_0 = 2^{35} \pmod{561} = 263$$

Compute $b_1 = b_0^2 \pmod{n}$

$$\begin{aligned} b_1 &= 263^2 \pmod{561} \\ 263^2 &= 69169 \end{aligned}$$

Now divide by 561:

$$\begin{aligned} 561 \times 123 &\Rightarrow 561 \times 100 = 56100 \\ 561 \times 20 &= 11220 \\ 561 \times 3 &= 1683 \end{aligned}$$

$$\begin{aligned} \text{Sum: } 56100 + 11220 &= 67320 \\ 67320 + 1683 &= 69003 \\ 69169 - 69003 &= 166 \end{aligned}$$

So:

$$b_1 = 166$$

Compute b_2

$$= b_1^2 \pmod{n}$$

$$\begin{aligned} b_2 &= 166^2 \pmod{561} \\ 166^2 &= 27556 \end{aligned}$$

Divide by 561:

$$\begin{aligned} 561 \times 49 &\Rightarrow 561 \times 50 \\ &= 28050 \\ 28050 - 561 &= 27489 \\ 27556 - 27489 &= 67 \end{aligned}$$

So:

$$b_2 = 67$$

Miller Rabin Test

Compute $b_3 = b_2^2 \pmod{n}$ $b_3 = 67^2 \pmod{561} = 4489 \pmod{561} = 1$

Sequence summary (we need to repeat up to max $b_k = b_4$)

$b_0 = 263, b_1 = 166, b_2 = 67, b_3 = 1$ (No need to compute b_4 , as $b_3 = 1$)

$n = 561$ is definitely composite.

$$561 = 3 \times 11 \times 17$$

Note:

In the Miller-Rabin test, if a sequence reaches 1 (e.g. in case of b_3) without the immediately preceding value being -1 (which is $560 \pmod{561}$), the number is composite.

Miller Rabin Test

Example Testing $n = 221$ for Primality

Step 1: Factor $n - 1$

$$n - 1 = 221 - 1 = 220$$

We write 220 as $2^k \cdot m$ with m odd:

Divide by 2 repeatedly:

$$220 \div 2^1 = 110 \text{ (Here, } k' = 1, m = 110) \text{ but } m \text{ is even, so continue till } m \text{ is odd}$$

$$220 \div 2^2 = 55 \text{ (} k' = 2, m = 55) \text{ } m \text{ is odd}$$

So:

$$n - 1 = 2^2 \cdot 55$$

$$k = 2, m = 55$$

Step 2: Select a Random Base

Choose a with $1 < a < 220$.

Let's pick $a = 4$ (randomly chosen).

Step 3: Compute Sequence of Powers

We compute:

$$b_0 = a^m \pmod{n}$$

$$b_0 = 4^{55} \pmod{221}$$

Miller Rabin Test

Let's compute using binary exponentiation:

Write 55 in binary: $55 = 32 + 16 + 4 + 2 + 1$

Or: $55 = 110111_2$

Build powers of 4 modulo 221:

1. $4^1 \pmod{221} = 4$

2. $4^2 = 16 \pmod{221} = 16$

3. $4^4 = 16^2 = 256$

$256 \div 221 = 1 \text{ remainder } 35$

So: $4^4 \pmod{221} = 35$

4. $4^8 = 35^2 = 1225$

$221 \times 5 = 1105$

$1225 - 1105 = 120$

So: $4^8 \pmod{221} = 120$

5. $4^{16} = 120^2 = 14400$

$221 \times 65 = 14365$

$14400 - 14365 = 35$

So: $4^{16} \pmod{221} = 35$

6. $4^{32} = 35^2 = 1225$

As before: $1225 \div 221 = 5 \text{ remainder } 120$

So: $4^{32} \pmod{221} = 120$

Now combine for 4^{55} :

$$\begin{aligned} 4^{55} &= 4^{32} \times 4^{16} \times 4^4 \times 4^2 \times 4^1 \\ &= 120 \times 35 \times 35 \times 16 \times 4 \pmod{221} \end{aligned}$$

Miller Rabin Test

Step-by-step multiplication (mod 221):

1. $120 \times 35 = 4200$

$$221 \times 19 = 4199$$

$$4200 - 4199 = 1$$

$$\text{So: } 120 \times 35 \equiv 1 \pmod{221}$$

2. $1 \times 35 = 35$

3. $35 \times 16 = 560$

$$221 \times 2 = 442$$

$$560 - 442 = 118$$

4. $118 \times 4 = 472$

$$221 \times 2 = 442$$

$$472 - 442 = 30$$

Thus:

$$b_0 = 4^{55} \pmod{221} = 30$$

Now , Compute $b_1 = b_0^2 \pmod{n}$

$$b_1 = 30^2 \pmod{221} = 900 \pmod{221}$$

$$221 \times 4 = 884$$

$$900 - 884 = 16$$

So:

$$b_1 = 16$$

Compute $b_2 = b_1^2 \pmod{n}$

$$b_2 = 16^2 \pmod{221} = 256 \pmod{221}$$

As calculated earlier: $256 \div 221$
 $= 1$ remainder 35

So:

$$b_2 = 35$$

Sequence summary:

$$b_0 = 30, b_1 = 16, b_2 = 35$$

Since **both conditions fail**, $n = 221$ is **definitely composite**.

Actually, we can factor it:

$221 = 13 \times 17$, confirming it's composite.